

# Kernel API

## Lab objectives

- Familiarize yourself with the basic Linux kernel API
- Description of memory allocation mechanisms
- Description of locking mechanisms

## Overview

Inside the current lab we present a set of concepts and basic functions required for starting Linux kernel programming. It is important to note that kernel programming differs greatly from user space programming. The kernel is a stand-alone entity that can not use libraries in user-space (not even libc). As a result, the usual user-space functions (printf, malloc, free, open, read, write, memcpy, strcpy, etc.) can no longer be used. In conclusion, kernel programming is based on a totally new and independent API that is unrelated to the user-space API, whether we refer to POSIX or ANSI C (standard C language library functions).

## Accessing memory

An important difference in kernel programming is how to access and allocate memory. Due to the fact that kernel programming is very close to the physical machine, there are important rules for memory management. First, it works with several types of memory:

- Physical memory
- Virtual memory from the kernel address space
- Virtual memory from a process's address space
- Resident memory - we know for sure that the accessed pages are present in physical memory

Virtual memory in a process's address space can not be considered resident due to the virtual memory mechanisms implemented by the operating system: pages may be swapped or simply may not be present in physical memory as a result of the demand paging mechanism. The memory in the kernel address space can be resident or not. Both the data and code segments of a module and the kernel stack of a process are resident. Dynamic memory may or may not be resident, depending on how it is allocated.

When working with resident memory, things are simple: memory can be accessed at any time. But if working with non-resident memory, then it can only be accessed from certain contexts. Non-resident memory can only be accessed from the process context. Accessing non-resident memory from the context of an interrupt has unpredictable results and, therefore, when the operating system detects such access, it will take drastic measures: blocking or resetting the system to prevent serious corruption.

The virtual memory of a process can not be accessed directly from the kernel. In general, it is totally discouraged to access the address space of a process, but there are situations where a device driver needs to do it. The typical case is where the device driver needs to access a buffer from the user-space. In this case, the device driver must use special features and not directly access the buffer. This is necessary to prevent access to invalid memory areas.

Another difference from the user-space scheduling, relative to memory, is due to the stack, a stack whose size is fixed and limited. A stack of 4K is used in Linux, and a stack of 12K is used in Windows. For this reason, the allocation of large structures on stack or the use of recursive calls should be avoided.

## Contexts of execution

In relation to kernel execution, we distinguish two contexts: process context and interrupt context. We are in the process context when we run code as a result of a system call or when we run in the context of a kernel thread. When we run in a routine to handle an interrupt or a deferrable action, we run in an interrupt context.

Some of the kernel API calls can block the current process. Common examples are using a semaphore or waiting for a condition. In this case, the process is put into the **WAITING** state and another process is running. An interesting situation occurs when a function that can lead to the current process to be suspended, is called from an interrupt context. In this case, there is no current process, and therefore the results are unpredictable. Whenever the operating system detects this condition will generate an error condition that will cause the operating system to shut down.

## Locking

One of the most important features of kernel programming is parallelism. Linux supports SMP systems with multiple processors and kernel preemptivity. This makes kernel programming more difficult because access to global variables must be synchronized with either spinlock primitives or blocking primitives. Although it is recommended to use blocking primitives, they can not be used in an interrupt context, so the only locking solution in the context of an interrupt is spinlocks.

Spinlocks are used in order to achieve mutual exclusion. When it can not get access to the critical region, it does not suspend the current process, but it uses the busy-waiting mechanism (waiting in a